Suman Sourav
Jonathan J. Huerta y Munive

AALBORG UNIVERSITY
DENMARK

Models and Tools for Cyber-Physical Systems
Class sheet 1
WITH SOLUTIONS

Exercise 1: Introduction to Feedback Control

A *controller* aims to regulate a physical quantity—the *process variable* (PV)—so that it reaches or maintains a desired value called the *set-point* (*SP*). Other common names for the set-point include *reference value* and *target value*.
A generic feedback control system consists of four main components: a **plant**, an **actuator**, a **sensor**, and a **controller**.

(a) Match each component name with its description.

Description	Component
(A) Measures the process variable and reports it back to the control loop	1. Plant
(B) The physical system or process being controlled	2. Actuator
(C) Computes the control signal based on the difference between the set-point and the measured process variable	3. Sensor
(D) Applies the control action to the plant (e.g. a motor driver, a valve)	4. Controller

(b) A control system is called *open-loop* if the control action is determined without using any feedback from the plant's output. It is called *closed-loop* if the controller receives measurements from a sensor and adjusts its output accordingly.

Classify each of the following systems as open-loop or closed-loop:

(i) A toaster that heats bread for a fixed amount of time, regardless of how toasted the bread is.

(ii) A thermostat that turns a heater on when the room temperature drops below a threshold and off when it exceeds it.

(iii) A washing machine that runs a pre-programmed cycle of fixed duration.

(iv) Cruise control in a car that measures the current speed and adjusts the throttle to maintain a target speed.

Solution sketch

(a) (A) $\rightarrow$ 3. Sensor; (B) $\rightarrow$ 1. Plant; (C) $\rightarrow$ 4. Controller; (D) $\rightarrow$ 2. Actuator.

(b) (i) Open-loop (no feedback on toasting level).

(ii) Closed-loop (temperature is measured and used to decide the control action).

(iii) Open-loop (fixed program, no feedback).

(iv) Closed-loop (speed is measured and the throttle is adjusted accordingly).

Exercise 2: Controlling a Robot's Position

A small robot moves along a straight line. Its position at time t is $x(t)$ and it starts at $x(0) = 0$. We want the robot to reach a target position SP by controlling its velocity $v(t)$. A proportional controller sets the velocity in proportion to how far the robot still is from the target.

(a) Formalise: "The robot's velocity equals the rate of change of its position."

(b) Formalise: "The controller sets the velocity proportionally to the error between the set-point and the current position, with proportionality constant K_p."

(c) Combine your answers from (a) and (b) to write the ordinary differential equation (ODE) that governs the robot's position under proportional control. State the initial condition.

(d) Solve this ODE. *Hint:* substitute $e(t) = SP - x(t)$.

(e) Find the equilibria of the system. Is the equilibrium asymptotically stable? Justify your answer.

Solution sketch

(a) $v(t) = \dot{x}(t)$.

(b) $v(t) = K_p \cdot (SP - x(t))$.

(c) Combining (a) and (b):

$$\dot{x}(t) = K_p \cdot (SP - x(t)), \qquad x(0) = 0.$$

(d) Let $e(t) = SP - x(t)$. Differentiating both sides gives $\dot{e}(t) = -\dot{x}(t)$. Since $\dot{x}(t) = K_p\, e(t)$ by the ODE from (c), we obtain

$$\dot{e}(t) = -K_p\, e(t), \qquad e(0) = SP - x(0) = SP - 0 = SP.$$

This is a standard first-order linear ODE whose solution is

$$e(t) = SP\, e^{-K_p\, t}.$$

Substituting back into $x(t) = SP - e(t)$:

$$x(t) \;=\; SP - SP\, e^{-K_p\, t} \;=\; SP \cdot \left(1 - e^{-K_p\, t}\right).$$

One can verify: $x(0) = SP(1-1) = 0$ ✓ and $\dot{x} = SP\, K_p\, e^{-K_p\, t} = K_p(SP - x)$ ✓.

(e) At equilibrium $\dot{x} = 0$, so $K_p(SP - x) = 0$, hence $x = SP$. Since $x(t) \to SP$ as $t \to \infty$ (for $K_p > 0$), the equilibrium is asymptotically stable.

Exercise 3: Controlling an Aircraft's Yaw Rate

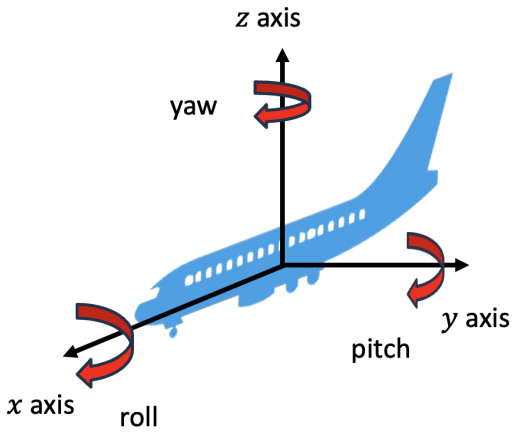


Figure 1: The three rotation axes of an aircraft: *yaw* (rotation around z—turning left or right), *pitch* (rotation around y—nose up or down), and *roll* (rotation around x—banking/tilting).

An aircraft can rotate around three axes (see Figure 1). The (simplified) objective of this exercise is to make the aircraft turn at a desired angular velocity r by applying a proportional controller.

(a) Which of the three angles (yaw, pitch, roll) is relevant when controlling left/right turning? Let us denote it by $\psi(t)$ and its angular velocity as $s(t) = \dot{\psi}(t)$.

The aircraft has moment of inertia $J > 0$ about the vertical axis. It is currently turning with angular velocity $s_0 \neq r$, and a proportional controller applies a corrective torque to bring the angular velocity to the set-point $SP = r$. Formalise each of the following statements.

(b) "The rate of change of the angular velocity equals the applied torque divided by the moment of inertia." *Hint:* this is the rotational analogue of Newton's second law.

(c) "The controller applies a torque proportional to the error between the desired angular velocity and the current angular velocity, with proportionality constant K_p."

(d) "The desired angular velocity is r (the set-point) and the aircraft initially turns with angular velocity s_0."

(e) Combine your answers from (b), (c), and (d) to write the ODE governing the angular velocity $s(t)$ under proportional control, together with the initial condition and the value of SP.

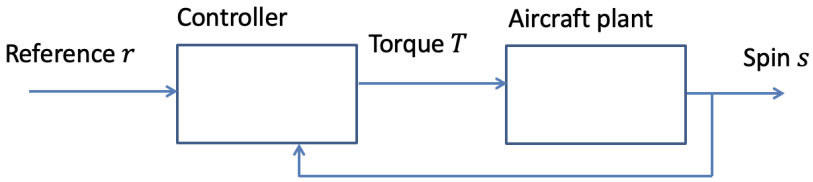


Figure 2: Block diagram of the proportional controller for the aircraft's yaw rate. Fill in the controller and plant equations.

(f) Complete the block diagram in Figure 2 by writing the corresponding equations in the **controller** and **plant** boxes.

(g) Solve this ODE.

(h) What happens if $K_p < 0$? Does the angular velocity converge to or diverge from the set-point?

(i) What happens if $K_p > 0$?

(j) What should one do to ensure faster convergence to the desired angular velocity?

.....Solution sketch .....

(a) The yaw angle controls the heading (turning left or right), so we control the yaw.

(b) $\dot{s}(t) = T(t)/J$.

(c) $T(t) = K_p \cdot (SP - s(t))$.

(d) $SP = r$ and $s(0) = s_0$.

(e) Substituting (c) into (b):

$$\dot{s}(t) \;=\; \frac{K_p \cdot (r - s(t))}{J}, \qquad s(0) = s_0.$$

(f) The completed block diagram is shown in Figure 3. The controller box contains $T = K_p \cdot (r - s)$ and the plant box contains $\dot{s} = T/J$.

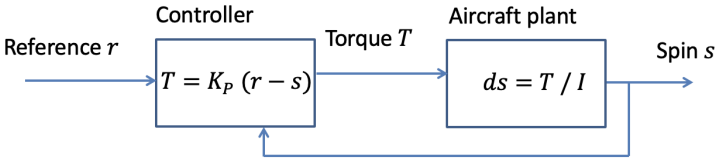


Figure 3: Completed block diagram.

(g) Let $e(t) = r - s(t)$. Then $\dot{e} = -\dot{s} = -K_p\, e/J$, so $e(t) = (r - s_0)\, e^{-K_p \cdot t/J}$. Substituting back:

$$s(t) \;=\; r - (r - s_0)\, e^{-K_p \cdot t/J}.$$

(h) If $K_p < 0$ the exponent $-K_p \cdot t/J > 0$ and the exponential grows as $t \to \infty$: $s(t)$ *diverges* from the set-point r.

(i) If $K_p > 0$ the exponent $-K_p \cdot t/J < 0$ and $s(t) \to r$ as $t \to \infty$. The angular velocity *converges* to the set-point.

(j) Increase K_p. A larger K_p increases the decay rate K_p/J, resulting in faster convergence.

Exercise 4: Modelling a DC Motor

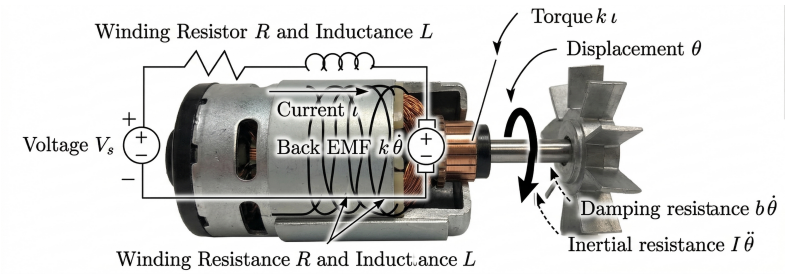


Figure 4: Electromechanical model of a DC motor. The electrical circuit represents the internal winding resistance (R) and inductance (L), while the mechanical side illustrates the conversion of current into torque to overcome rotor inertia and viscous damping.

We now study a more realistic control problem: regulating the rotational speed of a DC motor (for instance, one that drives a drone propeller). The goal is to derive the governing equations from first principles (see figure 4).

Electrical subsystem. Formalise each of the following physical statements as a mathematical equation.

(a) "The voltage across a resistor equals the product of the resistance R and the electrical current I flowing through the circuit."

(b) "The voltage across an inductor equals the product of the inductance L and the rate of change of the electrical current flowing through the circuit."

(c) "The electromotive-force (EMF) voltage generated by the rotating motor equals k times its angular velocity $\omega = \dot{\theta}$."

(d) "The input voltage V is a non-negative constant."

(e) "Kirchhoff's voltage law: the sum of voltages around a closed loop in an electrical circuit equals zero." Apply this to the motor circuit consisting of V, V_R, V_L, and V_M.

Mechanical subsystem. Formalise each of the following statements.

(f) "The resulting inertial torque equals the moment of inertia J times the angular acceleration." *Hint:* think of Newton's second law for rotation, where J plays the role of mass and θ the role of position.

(g) "The damping torque is proportional to the angular velocity, with proportionality constant b."

(h) "The torque produced by the motor is directly proportional to the motor's current via the EMF constant k."

(i) "The net torque acting on the motor shaft equals the motor torque minus the damping torque."

State-space representation.

(j) What is the quantity we would like to control in this system?

(k) Using the equations you derived, write the continuous-time state-space model of the DC motor with input voltage V and states ω (angular velocity) and I (current). A template is shown in Figure 5.

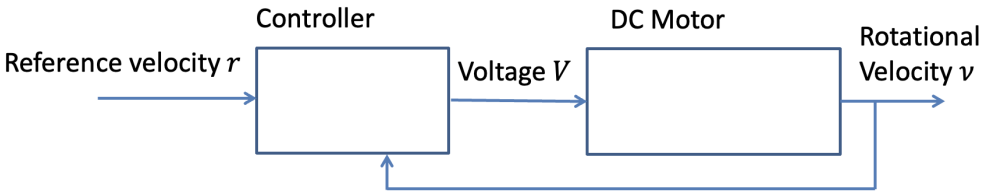


Figure 5: Fill this diagram with the appropriate equations.

.....Solution sketch .....

(a) $V_R = R \cdot I$

(b) $V_L = L \cdot \dot{I}$

(c) $V_M = k \cdot \omega = k \cdot \dot{\theta}$

(d) $V \geq 0$

(e) $V - V_R - V_L - V_M = 0$, i.e. $V = R\,I + L\,\dot{I} + k\,\omega$.

(f) $\tau = J\,\ddot{\theta} = J\,\dot{\omega}$

(g) $\tau_d = b \cdot \omega$

(h) $\tau_M = k \cdot I$

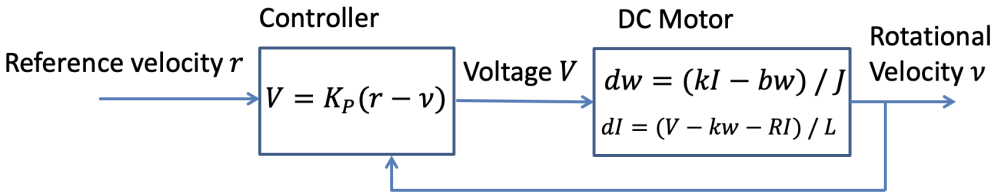
(i) $\tau = \tau_M - \tau_d$, i.e. $J\,\dot{\omega} = k\,I - b\,\omega$.

(j) We want to control the angular velocity ω of the motor (the rotational speed).

(k) From Kirchhoff's law: $L\,\dot{I} = V - k\,\omega - R\,I$. From Newton's second law for rotation: $J\,\dot{\omega} = k\,I - b\,\omega$. Hence the ODEs are

$$\dot{I} \;=\; \frac{V - k \cdot \omega - R \cdot I}{L}, \qquad \dot{\omega} \;=\; \frac{k \cdot I - b \cdot \omega}{J},$$

The continuous-time component diagram is shown below.



Exercise 5: PID Control of the DC Motor

We continue with the DC motor model from Exercise 4. The input voltage V is now determined by a controller, and the set-point is $SP = 1$ (rad/s). Use the default parameters $J = 0.01$, $b = 0.1$, $k = 0.01$, $R = 1$, $L = 0.5$.

Proportional control.

(a) A proportional controller sets $V = K_p \cdot (SP - \omega)$. Based on your experience from Exercises 2 and 3, do you expect the proportional controller to drive ω to the set-point? *Hint:* think about what the controller does once the plant *reaches* the desired speed, and what happens to the plant afterwards.

(b) Compute the equilibria of the closed-loop system. That is, set $\dot{I} = 0$ and $\dot{\omega} = 0$ and solve for I and ω in terms of K_p and the motor parameters.

(c) Evaluate the equilibrium angular velocity ω^* for $R = 1$, $b = 0.1$, $k = 0.01$. Is ω^* equal to SP?

(d) What happens if we increase K_p? Can we eliminate the steady-state error entirely by choosing K_p large enough?

(e) In physical terms, what is the key difference between this DC motor example and the simpler systems of Exercises 2 and 3 that produces this steady-state error?

Adding integral action.
The problem with the proportional controller is that it does not account for the accumulated past error. Below is pseudo-code for a (discretised) proportional controller:

```
LOOP
  PV := sensor_read()
  error := SP − PV
  u := K_p · error
```

(f) What modifications would you make to the pseudo-code above so that the controller keeps track of the *history* of the error? You may *not* use arrays or logging; instead, use a running sum variable. *Hint:* think about adding a term $K_i \cdot$ `error_sum`.

(g) *Using the provided simulation:* does the addition of integral action help the system reach the set-point? Why or why not?

(h) *Using the provided simulation:* what happens if you increase the constant K_i?

Step-response characteristics.

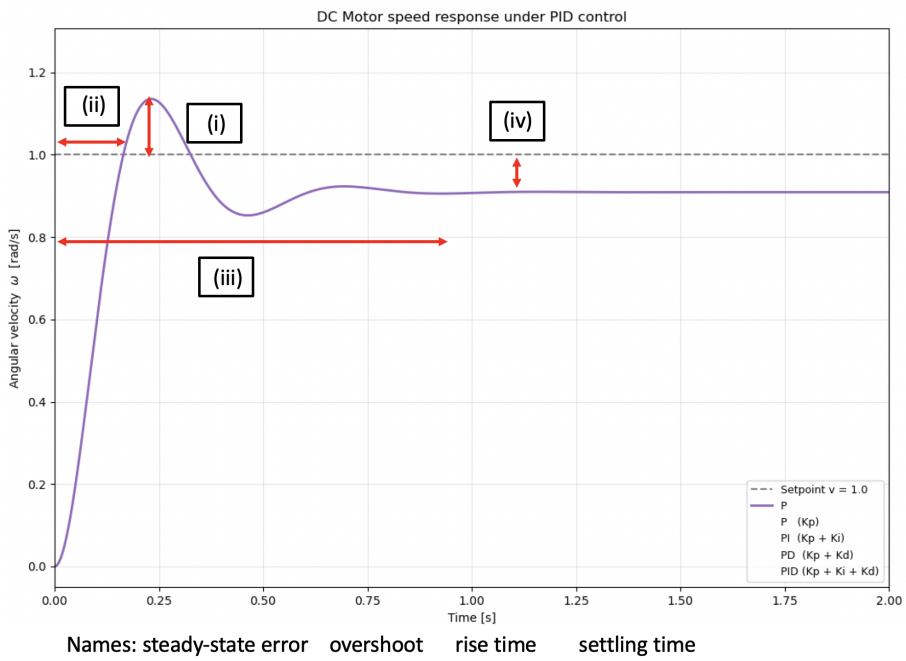


Figure 6: Step response of a controlled system. Match the labels (i)–(iv) to the names.

(i) Given the step-response plot in Figure 6, match the labels (i)–(iv) to the names in the figure.

(j) Match each definition on the left with the correct term on the right.

Definition	**Term**
(A) Difference between the maximum output value and the reference value	1. Rise time
(B) Time at which the output first crosses the reference value	2. Settling time
(C) Time at which the output reaches and stays within a band around the steady-state value	3. Overshoot
(D) Difference between the steady-state output value and the reference value	4. Steady-state error

(k) *Using the lecturer's projected simulation*, estimate the following for the current PI controller settings:

(i) percentage of overshoot,

(ii) rise time,

(iii) settling time,

(iv) steady-state error percentage.

Adding derivative action.

(l) Suppose the PI controller overshoots excessively. Can we reduce the overshoot by adjusting K_i? If we reduce K_i, what is the consequence for the settling time, and why?

(m) To reduce overshoot while keeping a relatively short settling time, we could look at how fast the error is *changing* over time—in a sense, "look into the future." What modifications would you make to your PI pseudo-code to incorporate this rate of change? (Hint: the continuous version is presented in Figure 7.)

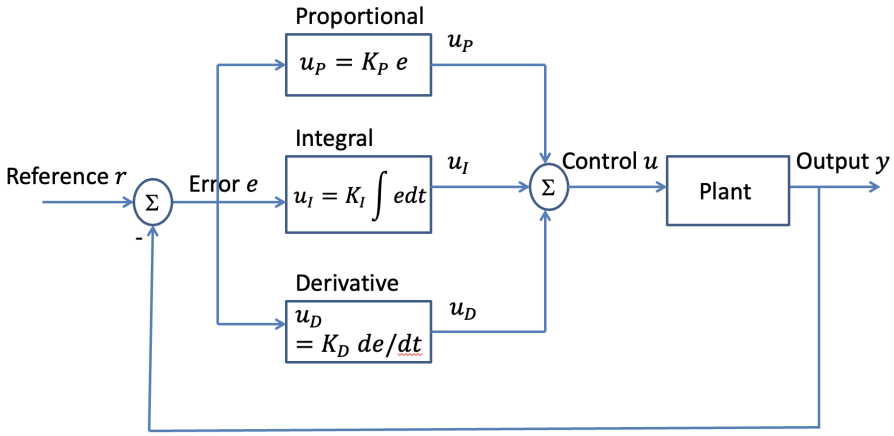


Figure 7: Generic representation of a PID controller.

.....Solution sketch .....

(a) Unlike Exercises 2 and 3, the DC motor involves additional dynamics: inductance, resistance, and back-EMF. When ω reaches SP, the error is zero, so $V = 0$. However, with no voltage applied, friction ($b\,\omega$) and back-EMF slow the motor down. Hence the proportional controller alone cannot maintain the desired speed—there will be a steady-state error.

(b) At equilibrium ($\dot{I} = 0$, $\dot{\omega} = 0$):

$$0 = k\,I - b\,\omega \qquad \Longrightarrow \quad I = \frac{b\,\omega}{k},$$

$$0 = K_p(1-\omega) - k\,\omega - R\,I = K_p(1-\omega) - k\,\omega - \frac{R\,b\,\omega}{k}.$$

Solving for ω:

$$K_p \;=\; \omega\left(K_p + k + \frac{R\,b}{k}\right) \quad \Longrightarrow \quad \omega^* = \frac{k\,K_p}{k\,K_p + k^2 + R\,b}, \qquad I^* = \frac{b\,K_p}{k\,K_p + k^2 + R\,b}.$$

(c) With $R = 1$, $b = 0.1$, $k = 0.01$:

$$\omega^* = \frac{0.01\,K_p}{0.01\,K_p + 0.0001 + 0.1} = \frac{0.01\,K_p}{0.01\,K_p + 0.1001}.$$

For example, $K_p = 100$ gives $\omega^* \approx 0.909$, which is strictly less than $SP = 1$. There is a steady-state error.

(d) Increasing K_p brings ω^* closer to 1, but $\omega^* < 1$ for any finite K_p. One would need $K_p \to \infty$ to eliminate the error entirely, which is physically unrealisable.

(e) In Exercises 2 and 3 the control input acted directly on the quantity we wanted to control (position or angular velocity). In the DC motor the controller sets the voltage V, which drives the current I, which in turn produces torque. The additional dynamics (resistance, inductance, back-EMF) introduce losses that prevent the proportional controller from eliminating the steady-state error.

(f) The modified pseudo-code with integral action:

```
error_sum := 0
prev_time := 0
LOOP
  PV, time := sensor_read()
  dt := time − prev_time
  error := SP − PV
  error_sum := error_sum + error
  u := K_p · error + K_i · error_sum · dt
  prev_time := time
```

(g) Yes. The integral term accumulates past error, so even a small persistent error eventually produces a large enough control signal to close the gap. The PI controller eliminates the steady-state error.

(h) Increasing K_i makes the controller react more aggressively to accumulated error. This reduces the settling time but increases overshoot and can cause oscillations if K_i is too large.

(i) The four labels are: (i) Overshoot, (ii) Rise time, (iii) Settling time, (iv) Steady-state error.

(j) (A) → 3. Overshoot; (B) → 1. Rise time; (C) → 2. Settling time; (D) → 4. Steady-state error.

(k) *Answers depend on the simulation parameters used by the lecturer during the session.*

(l) Yes, reducing K_i decreases overshoot. However, the trade-off is that the settling time *increases*, because the integral term accumulates error more slowly and therefore takes longer to correct the remaining offset.

(m) We add a derivative term proportional to the rate of change of the error. The modified pseudo-code becomes:

$$
\begin{aligned}
&\texttt{error\_sum} := 0 \\
&\texttt{prev\_time} := 0 \\
&\texttt{prev\_error} := 0 \\
&\texttt{LOOP} \\
&\quad \texttt{PV}, \texttt{time} := \texttt{sensor\_read()} \\
&\quad \texttt{dt} := \texttt{time} - \texttt{prev\_time} \\
&\quad \texttt{error} := SP - \texttt{PV} \\
&\quad \texttt{error\_sum} := \texttt{error\_sum} + \texttt{error} \\
&\quad \texttt{u} := K_p \cdot \texttt{error} + K_i \cdot \texttt{error\_sum} \cdot \texttt{dt} + K_d \cdot \frac{\texttt{error} - \texttt{prev\_error}}{\texttt{dt}} \\
&\quad \texttt{prev\_time} := \texttt{time} \\
&\quad \texttt{prev\_error} := \texttt{error}
\end{aligned}
$$